Code Anaylsis:

a. Name of two GenAI programs: Claude and Google Gemini

b. How you accessed them (e.g., Google query, CoPilot on my computer)
Claude was accessed through the Claude website on my computer and Gemini was accessed through the Google Gemini website on my computer.

c. Prompt used. (You must use the same prompt for both GenAIs.)

generate C++ code using objects not functions implement
the following requirements
• The program will prioritize emails for a busy company CEO.
• You will use a MaxHeap as a means of implementing a priority queue. A priority queue is a queue where emails can shift towards the front of the queue based on a priority status.
• You must implement a MaxHeap using a list-based implementation. Then use that MaxHeap to handle all your email prioritizing for the CEO.
• You must create objects from scratch. Do not include pre-existing heap modules.
• Test File format:
Command Description
EMAIL <sender category>,
<subject line>,
<date>
The emails in the CEO's Inbox should be placed in queue based on their sender category and date. The sender categories and priority to be read are as follows:
• Boss – read first
• Subordinate – read next
• Peer – read next
• ImportantPerson – read next
• OtherPerson – read last
If there is more than one from a sender, then the newest email (not the oldest) should be read first. I discovered this trick while a manager at Sprint.
EMAIL is followed by space. The rest of the fields are delimited.
Assume <sender category> is one of the five strings listed above.
Assume <subject line> is a string which may contain spaces,

but not commas
Assume <date> is in the format: MM-DD-YYYY
NEXT
Next email for the CEO to read. Display the information on
the terminal in the following format:
Sender:
Subject:
Date:
READ CEO has read email and dealt with it
COUNT Display current count of unread emails
• The grader will test your code with a different test file. The test file will begin with the emails in the CEOs Inbox, followed by the commands the CEO will use to read her Inbox. You can assume a given command will be formatted as shown, but the order and number of commands may vary. For example, be able to handle a file that does NEXT or READ when there are no emails, two NEXTs in a row without an intervening READ (output should be the same for both NEXTs), two READs in a row without an intervening NEXT (highest priority email deleted from queue without displaying it).
• Sample Test File:
EMAIL Peer,Can you help me on this?,12-01-2024
EMAIL OtherPerson,Try our product,12-19-2024
EMAIL Boss,Important,12-20-2024
EMAIL Subordinate,How do I handle this?,12-25-2024
EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024
EMAIL Boss,Never Mind,01-03-2025
COUNT
NEXT
READ
NEXT
READ
COUNT
• Your program should display this:
There are 6 emails to read.
Next email:
Sender: Boss
Subject: Never Mind
Date: 01-03-2025
Next email:
Sender: Boss
Subject: Important

Date: 12-20-2024
There are 4 emails to read.

d. Generated code from both GenAIs.
Claude:

```
// CEO Email Prioritizer - object-oriented C++ with a hand-built
array-based MaxHeap.
// Usage: ./ceo_email [testfile]   (reads stdin if no file is given)

#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
using namespace std;

// ---------------------------------------------------------------- Email
class Email {
private:
   string sender;
   string subject;
   string date;      // MM-DD-YYYY
   int    rank;      // higher = read sooner
   int    dateKey;   // YYYYMMDD, higher = newer
   long   sequence;  // arrival order, used only to break exact ties

   static int rankFor(const string& s) {
       if (s == "Boss")            return 5;
       if (s == "Subordinate")     return 4;
       if (s == "Peer")            return 3;
       if (s == "ImportantPerson") return 2;
       return 1;                   // OtherPerson
   }

   static int keyFor(const string& d) {
       // MM-DD-YYYY -> YYYYMMDD
       int mm = atoi(d.substr(0, 2).c_str());
       int dd = atoi(d.substr(3, 2).c_str());
       int yy = atoi(d.substr(6, 4).c_str());
       return yy * 10000 + mm * 100 + dd;
   }
```

```
public:
    Email() : rank(0), dateKey(0), sequence(0) {}
    Email(const string& s, const string& subj, const string& d, long seq)
        : sender(s), subject(subj), date(d),
          rank(rankFor(s)), dateKey(keyFor(d)), sequence(seq) {}

    // True if this email should be read before 'other'.
    bool higherPriorityThan(const Email& other) const {
        if (rank != other.rank)       return rank > other.rank;
        if (dateKey != other.dateKey) return dateKey > other.dateKey; //
newest first
        return sequence < other.sequence;                              //
earlier arrival
    }

    void display() const {
        cout << "Sender: "  << sender  << "\n"
             << "Subject: " << subject << "\n"
             << "Date: "    << date    << "\n";
    }
};

// ------------------------------------------------------------ MaxHeap
class MaxHeap {
private:
    Email* data;
    int    size;
    int    capacity;

    static int parent(int i) { return (i - 1) / 2; }
    static int left(int i)   { return 2 * i + 1; }
    static int right(int i)  { return 2 * i + 2; }

    void swapAt(int a, int b) {
        Email tmp = data[a];
        data[a] = data[b];
        data[b] = tmp;
    }
```

```
    void grow() {
        int newCap = capacity * 2;
        Email* bigger = new Email[newCap];
        for (int i = 0; i < size; i++) bigger[i] = data[i];
        delete[] data;
        data = bigger;
        capacity = newCap;
    }

    void siftUp(int i) {
        while (i > 0 && data[i].higherPriorityThan(data[parent(i)])) {
            swapAt(i, parent(i));
            i = parent(i);
        }
    }

    void siftDown(int i) {
        while (true) {
            int best = i, l = left(i), r = right(i);
            if (l < size && data[l].higherPriorityThan(data[best])) best =
l;
            if (r < size && data[r].higherPriorityThan(data[best])) best =
r;
            if (best == i) break;
            swapAt(i, best);
            i = best;
        }
    }

public:
    MaxHeap() : data(new Email[16]), size(0), capacity(16) {}
    ~MaxHeap() { delete[] data; }
    MaxHeap(const MaxHeap&) = delete;
    MaxHeap& operator=(const MaxHeap&) = delete;

    bool isEmpty() const { return size == 0; }
    int  count()   const { return size; }

    void insert(const Email& e) {
        if (size == capacity) grow();
```

```
        data[size] = e;
        siftUp(size);
        size++;
    }

    const Email& peekMax() const { return data[0]; }   // caller checks
isEmpty()

    void removeMax() {                                  // caller checks
isEmpty()
        data[0] = data[size - 1];
        size--;
        if (size > 0) siftDown(0);
    }
};

// ------------------------------------------------------------ CEOInbox
class CEOInbox {
private:
    MaxHeap queue;
    long    nextSequence;

public:
    CEOInbox() : nextSequence(0) {}

    void addEmail(const string& sender, const string& subject, const
string& date) {
        queue.insert(Email(sender, subject, date, nextSequence++));
    }

    void next() const {
        if (queue.isEmpty()) {
            cout << "No emails to read.\n";
            return;
        }
        cout << "Next email:\n";
        queue.peekMax().display();
    }

    void read() {
```

```
        if (!queue.isEmpty()) queue.removeMax();
    }

    void count() const {
        cout << "There are " << queue.count() << " emails to read.\n";
    }
};

// ------------------------------------------------------ CommandProcessor
class CommandProcessor {
private:
    CEOInbox& inbox;

    static string trim(const string& s) {
        size_t a = s.find_first_not_of(" \t\r\n");
        if (a == string::npos) return "";
        size_t b = s.find_last_not_of(" \t\r\n");
        return s.substr(a, b - a + 1);
    }

    void handleEmail(const string& args) {
        // <sender category>,<subject line>,<date>
        size_t c1 = args.find(',');
        size_t c2 = args.rfind(',');
        if (c1 == string::npos || c2 == c1) return;   // malformed, ignore
        string sender  = trim(args.substr(0, c1));
        string subject = trim(args.substr(c1 + 1, c2 - c1 - 1));
        string date    = trim(args.substr(c2 + 1));
        inbox.addEmail(sender, subject, date);
    }

public:
    explicit CommandProcessor(CEOInbox& in) : inbox(in) {}

    void process(istream& in) {
        string line;
        while (getline(in, line)) {
            line = trim(line);
            if (line.empty()) continue;
```

```
            if (line.compare(0, 6, "EMAIL ") == 0)
handleEmail(line.substr(6));
            else if (line == "NEXT")                inbox.next();
            else if (line == "READ")                inbox.read();
            else if (line == "COUNT")               inbox.count();
        }
    }
};

// ------------------------------------------------------------------ main
int main(int argc, char* argv[]) {
    CEOInbox inbox;
    CommandProcessor processor(inbox);

    if (argc > 1) {
        ifstream file(argv[1]);
        if (!file) {
            cerr << "Could not open file: " << argv[1] << "\n";
            return 1;
        }
        processor.process(file);
    } else {
        processor.process(cin);
    }
    return 0;
}
```

Google Gemini:

```
#include <iostream>
#include <string>
#include <sstream>
#include <iomanip>
#include <fstream>

// Date representation to allow chronological comparison
class Date {
private:
    int month;
    int day;
    int year;
```

```
public:
    Date() : month(0), day(0), year(0) {}

    // Parses string in "MM-DD-YYYY" format
    Date(const std::string& dateStr) {
        std::stringstream ss(dateStr);
        char dash1, dash2;
        ss >> month >> dash1 >> day >> dash2 >> year;
    }

    // Modern dates are strictly greater
    bool operator>(const Date& other) const {
        if (year != other.year) return year > other.year;
        if (month != other.month) return month > other.month;
        return day > other.day;
    }

    bool operator==(const Date& other) const {
        return year == other.year && month == other.month && day ==
other.day;
    }

    std::string toString() const {
        std::ostringstream ss;
        ss << std::setfill('0') << std::setw(2) << month << "-"
           << std::setfill('0') << std::setw(2) << day << "-"
           << std::setw(4) << year;
        return ss.str();
    }
};

// Represents an individual Email
class Email {
private:
    std::string senderCategory;
    std::string subject;
    Date date;
```

```
    // Helper mapping sender categories to numerical rank (Higher = More
Important)
    int getCategoryPriority() const {
        if (senderCategory == "Boss") return 5;
        if (senderCategory == "Subordinate") return 4;
        if (senderCategory == "Peer") return 3;
        if (senderCategory == "ImportantPerson") return 2;
        if (senderCategory == "OtherPerson") return 1;
        return 0;
    }

public:
    Email() = default;

    Email(std::string sender, std::string subj, std::string dStr)
        : senderCategory(sender), subject(subj), date(dStr) {}

    std::string getSenderCategory() const { return senderCategory; }
    std::string getSubject() const { return subject; }
    std::string getDateString() const { return date.toString(); }

    // Compares two emails based on requirements:
    // 1. Sender category priority
    // 2. Newer date first if category matches
    bool operator>(const Email& other) const {
        int thisPri = getCategoryPriority();
        int otherPri = other.getCategoryPriority();

        if (thisPri != otherPri) {
            return thisPri > otherPri;
        }
        return date > other.date;
    }

    bool operator<(const Email& other) const {
        return other > *this;
    }
};
```

```cpp
// Custom dynamic list structure to back the MaxHeap without built-in
std::vector or containers
template <typename T>
class DynamicList {
private:
    T* data;
    int capacity;
    int currentSize;

    void resize() {
        capacity = (capacity == 0) ? 10 : capacity * 2;
        T* newData = new T[capacity];
        for (int i = 0; i < currentSize; ++i) {
            newData[i] = data[i];
        }
        delete[] data;
        data = newData;
    }

public:
    DynamicList() : data(nullptr), capacity(0), currentSize(0) {}

    ~DynamicList() {
        delete[] data;
    }

    void push_back(const T& val) {
        if (currentSize == capacity) {
            resize();
        }
        data[currentSize++] = val;
    }

    void pop_back() {
        if (currentSize > 0) {
            --currentSize;
        }
    }

    T& operator[](int index) {
```

```
        return data[index];
    }

    const T& operator[](int index) const {
        return data[index];
    }

    int size() const {
        return currentSize;
    }

    bool empty() const {
        return currentSize == 0;
    }
};

// Custom MaxHeap implementation built on top of the dynamic list
template <typename T>
class MaxHeap {
private:
    DynamicList<T> heapList;

    void heapifyUp(int index) {
        while (index > 0) {
            int parentIndex = (index - 1) / 2;
            if (heapList[index] > heapList[parentIndex]) {
                T temp = heapList[index];
                heapList[index] = heapList[parentIndex];
                heapList[parentIndex] = temp;
                index = parentIndex;
            } else {
                break;
            }
        }
    }

    void heapifyDown(int index) {
        int size = heapList.size();
        while (index < size) {
            int leftChild = 2 * index + 1;
```

```
            int rightChild = 2 * index + 2;
            int largest = index;

            if (leftChild < size && heapList[leftChild] >
heapList[largest]) {
                largest = leftChild;
            }
            if (rightChild < size && heapList[rightChild] >
heapList[largest]) {
                largest = rightChild;
            }

            if (largest != index) {
                T temp = heapList[index];
                heapList[index] = heapList[largest];
                heapList[largest] = temp;
                index = largest;
            } else {
                break;
            }
        }
    }

public:
    MaxHeap() = default;

    void insert(const T& value) {
        heapList.push_back(value);
        heapifyUp(heapList.size() - 1);
    }

    T getMax() const {
        if (heapList.empty()) {
            throw std::runtime_error("Heap is empty");
        }
        return heapList[0];
    }

    void removeMax() {
        if (heapList.empty()) return;
```

```
        heapList[0] = heapList[heapList.size() - 1];
        heapList.pop_back();
        if (!heapList.empty()) {
            heapifyDown(0);
        }
    }

    int size() const {
        return heapList.size();
    }

    bool empty() const {
        return heapList.empty();
    }
};

// Email Manager handles high-level operations for the CEO's Inbox
class CEOInboxManager {
private:
    MaxHeap<Email> emailHeap;

public:
    void addEmail(const std::string& category, const std::string& subject,
const std::string& date) {
        Email email(category, subject, date);
        emailHeap.insert(email);
    }

    void showCount() const {
        std::cout << "There are " << emailHeap.size() << " emails to read."
<< std::endl;
    }

    void showNext() const {
        if (emailHeap.empty()) {
            return;
        }
        Email nextEmail = emailHeap.getMax();
        std::cout << "Next email:" << std::endl;
```

```
        std::cout << "Sender: " << nextEmail.getSenderCategory() <<
std::endl;
        std::cout << "Subject: " << nextEmail.getSubject() << std::endl;
        std::cout << "Date: " << nextEmail.getDateString() << std::endl;
    }

    void readEmail() {
        if (!emailHeap.empty()) {
            emailHeap.removeMax();
        }
    }

    void processCommand(const std::string& line) {
        if (line.empty()) return;

        if (line.rfind("EMAIL", 0) == 0) {
            std::string content = line.substr(6); // Trim "EMAIL "
            std::stringstream ss(content);
            std::string category, subject, date;

            std::getline(ss, category, ',');
            std::getline(ss, subject, ',');
            std::getline(ss, date);

            addEmail(category, subject, date);
        } else if (line == "COUNT") {
            showCount();
        } else if (line == "NEXT") {
            showNext();
        } else if (line == "READ") {
            readEmail();
        }
    }
};

int main() {
    CEOInboxManager inbox;
    std::string line;

    // Process commands line-by-line from standard input
```

```
    while (std::getline(std::cin, line)) {
        inbox.processCommand(line);
    }

    return 0;
}
```

e. Analyze the code in terms of correctness, execution time, space complexity, and maintainability as explained in the "GenAI Code Comparison" lecture.

Correctness :

The raw code that Gemini generated based on the prompt it received behaved has expected with input files that were formatted as expected. However the error handling was mediocre. The program still processed emails with incorrect command lines or invalid inputs in the EMAIL command line. In addition to that, no message was generated when empty input files were entered and the program could not handle a no file input. The Claude program functioned exactly the same as the Gemini code with the same error handling mistakes.

Execution Time:

Both the Gemini and Claude code have similar time complexity because they both use a MaxHeap. Adding or removing an email takes O(log n), while viewing the next email or checking the count takes O(1). Overall, both are about O(n log n) when processing many emails. The Claude version may be slightly faster because it converts dates into numbers for easier comparison.

Space Complexity:

Both the Gemini code and the Claude code have a space complexity of O(n), where n is the number of emails being stored in the MaxHeap. This is because both programs use a dynamically allocated array that grows as more emails are added. The Gemini code stores each email with a sender category, subject, and a Date object. The Date object stores the month, day, and year as integers. The Claude code stores a little more information for each email, including the sender, subject, date string, priority rank, date key, and sequence number. Because of this, the Claude version may use slightly more memory for each email, but both programs still have the same overall space complexity of O(n). The extra variables used during heap operations only take up a constant amount of memory, so they do not change the overall space complexity.

Maintainability – ChatGPT:
Both the generated codes have minimal comments, nowhere near a comment on every line. However, the Claude generated code has slightly more comments than the Gemini generated code. The Claude generated code is also about 70 lines less than the Gemini generated code and more direct/organized.

f. Select one to use as the basis for this assignment and justify why you picked it.

I am going to use the Claude generated code as the basis for this assignment. Both programs are generally on par with each other with the same error handling. Space and Time complexity are very similar between the two programs. The Claude program might runs slightly faster but analysis wise it sort of balances because the Claude program uses slightly more memory. The reason why am choosing to continue modifying the Claude generated code boils down to maintainability. The Claude generated code is organzied in a way that would be easier to come back to and understand than the Gemini generate code that spreads out some of the logic.

g. Explain what you did to improve it in terms of correctness, execution time, space complexity, and maintainability.

I made a few changes to the Claude code mainly to make it more correct and easier to use. I added validation for the sender categories so random or misspelled categories would not automatically be treated like an OtherPerson. I also added date validation so the program only accepts dates in the correct MM-DD-YYYY format and makes sure the date is actually possible.

I added more error handling for things like invalid commands, missing subjects, malformed EMAIL commands, files that cannot be opened, and empty input files. I also made sure NEXT and READ work correctly when there are no emails left instead of risking an error.

I changed the input handling so the program can work either with a filename or with input redirection. This makes it more flexible depending on how the program is being tested. I also made empty lines get ignored since they do not really need to count as errors.

Lastly, I changed the output formatting so there is a blank line between error messages, NEXT outputs, and COUNT outputs. This does not change how the program works, but it makes the output a lot easier to read and follow. I also added comments to every line.